How do you use this module system? By creating files to be the modules and then joining them together with `require` and `exports`.

In this section, you'll learn to create, require, and export modules, why module names are not filenames, and how the module cache works. Finally, you'll see several listings presenting a complete program made up of multiple modules. First, you'll examine creating and requiring modules.

12.1.1. Creating and requiring

The first problem you and Scruffy encounter is that the `Blog` class `extends` the `Controller` class. When both classes are in the application.coffee file, everything works fine:

```
class Controller

class Blog extends Controller
```

There are no errors when compiling:

```
coffee –c application.coffee
```

But once you split those classes into separate files according to listing 12.1, then blog.coffee doesn't compile:

```
# -- blog.coffee --

class Blog extends Controller
```

You get a reference error:

```
coffee –c blog.coffee
# ReferenceError: Controller is not defined
```

You need some way for one file in your application to load other files. The built-in Node.js module system can do that. It's a good place to start looking at modules. To use the parts of the application contained in controller.coffee from inside blog.coffee, you `require` it with the module system: